UMEÅ UNIVERSITY
Department of computing science
Assignment 1

April 22, 2026

Formal and Cognitive Reasoning 5DV246

Deliverable 1 -Argument builder

Name	**E-mail**
Elias Niko	c18eno@cs.umu.se

Teacher
Juan Carlos Nieves

Contents

1 Introduction 1

2 Framework 1

3 Building arguments 1

4 Testing 2

1 Introduction

In this report we describe the argument builder implementing which is done for deliverable 1 part of the assignment. The objective is to impement an argument bilder that based on a knowledge base, produce a set of arguments. The argument builder is implemented in Python concidering a content-moderation reasoning domain.

2 Framework

The argumentation framewrok that is used for building the argument builder is assumption-based argumentation framework (ABA). An ABA framework has four parts. A language of atoms, a set of rules, a set of assumptions, and a contrary function that says, for each assumption, which atom would overturn it. An argument is a chain of reasoning from some assumptions down to a claim. An argument attacks another argument when its claim is the contrary of one of the other argument's assumptions.

For the spam/content moderation system, ABA seems to be the better option because it only has a single type of attack, which keeps the framework simple. The principle of starting from default assumptions and then identifying evidence that overturns them aligns with how a moderation system actually works. In this approach the system initially assumes users are acting legitimately unless there are contrary indications. The attack relation also falls out of the argument structure, which keep salgorithms short. For instance, checking whether one argument attacks another reduces to a single test. Is the attacker's claim the contrary of some assumption the target relies on? In ASPIC+ the same check would need to distinguish between rebut, undermine, and undercut attacks, each with its own conditions.

3 Building arguments

The `parser()` method reads the file once and returns a set of assumptions, a dictionary from each assumption to its contrary, and a list of rules represented as `(claim, bodys)` pairs.

The `build_argument()` method returns every argument that concludes `claim`. It handles three cases:

- If the claim is itself an assumption, it supports itself. The argument has the claim as its only supporting assumption.
- If there is a rule with an empty body, it produces an argument with no supporting assumptions.
- If there is a rule with a non-empty body, the function calls itself in a recursive way to find arguments in the body. If any body has zero arguments supporting it, the rule can't fire. Otherwise, it takes every combination of one sub-argument per body and builds a new argument whose support is the union of the sub-supports.

After collecting all the arguments, the function removes duplicates and if two derivations end up with the same support and the same claim, they count as the same argument.

The outer function `build_all_argument()` collects every statement that shows up anywhere in the knowledge base and calls `build_arguments()` on each one.

Once all the arguments are built, `find_defeats()` looks for attacks. For example argument A defeats argument B if A's claim is the contrary of some assumption in B's support.

The function loops over every pair of arguments and checks this condition. The `break` inside the inner loop makes sure each attacking pair gets recorded only once, even if the target has multiple assumptions that share the same contrary.

4 Testing

To test the builder, a small knowledge base for a content moderation scenario was written. The system starts with three default assumptions: the user is legitimate, the post is in good faith, and nothing looks like spam. Each default has a contrary that would overturn it. A top rule says to allow a post when all three defaults hold. The rest of the rules describe how evidence combines into those contraries.

Input - test 1

```
assumption a
assumption b

contrary(a) = x
contrary(b) = y

f :-
p :- a
q :- a, b
x :- f
```

Output

```
argument_builder$ python3 argument_builder.py test1.txt
Assumptions: ['a', 'b']
Contraries:  {'a': 'x', 'b': 'y'}

rules:
f :-
p :- a
q :- a, b
x :- f

Arguments: 6
A1: (a |- a)
A2: (b |- b)
A3: ( |- f)
A4: (a |- p)
A5: (a, b |- q)
A6: ( |- x)

Defeats: 3
A6 defeats A1
A6 defeats A4
A6 defeats A5
argument_builder$
```

Input - test 2 domain based KB

```
assumption legit_user
assumption good_looking_post
assumption no_spam_signals

contrary(legit_user) = confirmed_bot
contrary(good_looking_post) = harmful_intent
contrary(no_spam_signals) = spam_detected

allow :- legit_user, good_looking_post, no_spam_signals
confirmed_bot :- failed_captcha, exceeds_rate_limit
spam_detected :- known_spam

failed_captcha :-
exceeds_rate_limit :-
```

Output

```
argument_builder$ python3 argument_builder.py test2.txt
Assumptions: ['good_looking_post', 'legit_user', 'no_spam_signals']
Contraries:  {'legit_user': 'confirmed_bot', 'good_looking_post': 'harmful_intent', 'no_spam_signals': 'spam_detected'}

rules:
allow :- legit_user, good_looking_post, no_spam_signals
confirmed_bot :- failed_captcha, exceeds_rate_limit
spam_detected :- known_spam
failed_captcha :-
exceeds_rate_limit :-

Arguments: 7
A1: (good_looking_post, legit_user, no_spam_signals |- allow)
A2: ( |- confirmed_bot)
A3: ( |- exceeds_rate_limit)
A4: ( |- failed_captcha)
A5: (good_looking_post |- good_looking_post)
A6: (legit_user |- legit_user)
A7: (no_spam_signals |- no_spam_signals)

Defeats: 2
A2 defeats A1
A2 defeats A6
argument_builder$
```